

INTRODUÇÃO À ANÁLISE DE DADOS EM FÍSICA DE ALTAS ENERGIAS

Exercício Analise Python Colunar

Professores: Dilson de Jesus Damiano, Mauricio Thiel e Eliza Melo

Aluno: Thiago Henrique de Sousa

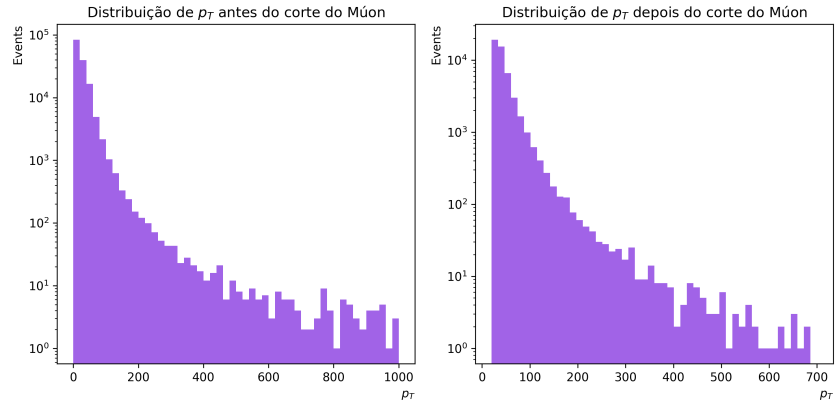
(26/11/2024)

Exercício

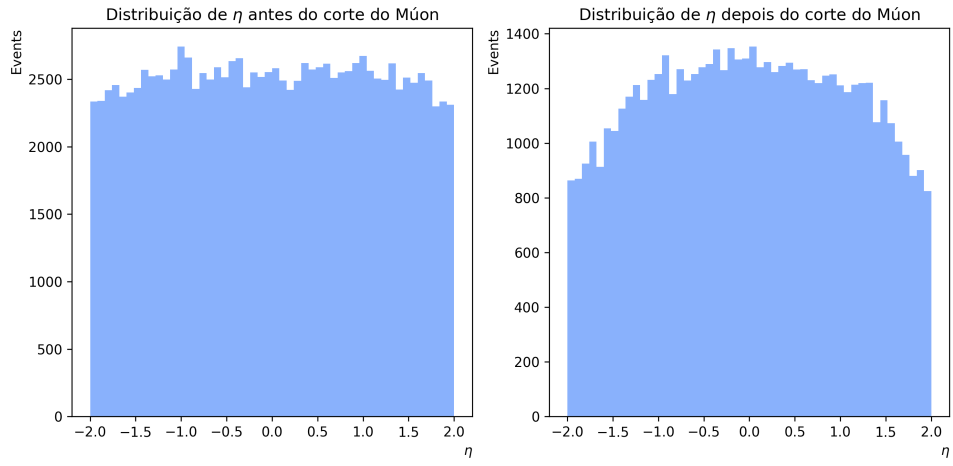
Neste exercício, aplicamos cortes de seleção no momento transversal, p_T , e no ângulo η nos léptons (múon, elétrons, taus) e nos jatos. Para todos, selecionamos que tenham 2 ou mais léptons ou jatos em cada evento e aplicamos os seguintes cortes, em GeV:

- Primeiro evento: $p_T > 20$ e $\eta < 2.5$
- Segundo evento: $p_T > 12$ e $\eta < 2.5$

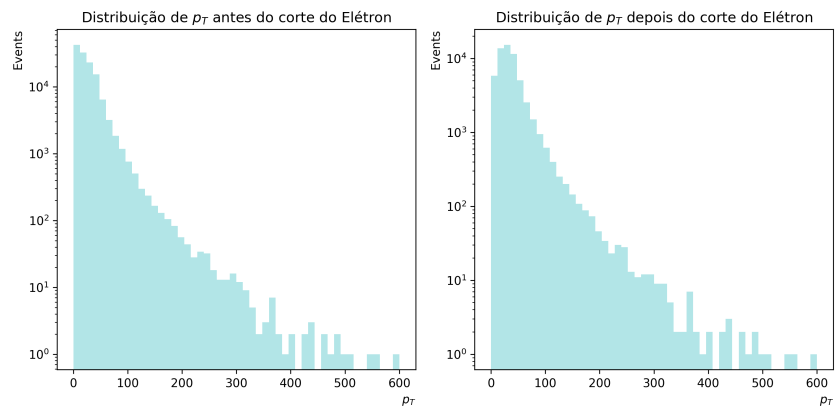
Esses cortes foram aplicados em todos os objetos utilizados na tarefa. Assim obtemos os gráficos do momento transversal e η para cada objeto e ao compara-los, observamos que há uma diminuição no número de eventos. O que é esperado, pois aplicamos cortes e exigimos que somente o evento que esteja dentro do intervalo requisitado seja selecionado. Assim obtemos os gráficos de p_T e η antes e depois do corte para múons, elétrons, taus e jatos respectivamente. E por fim, conseguimos o gráfico da massa através da soma dos quadri-vetores de cada objeto. Após a sequência dos gráficos, segue o código da tarefa.



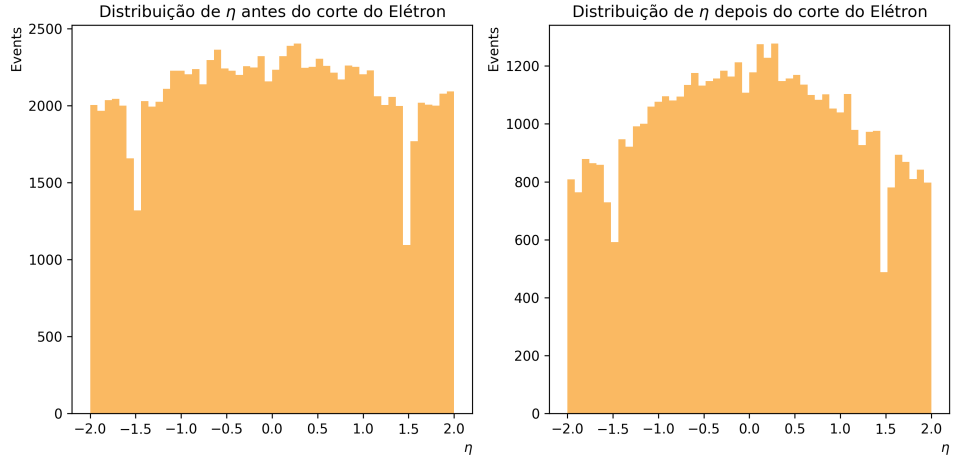
Fonte: O autor.



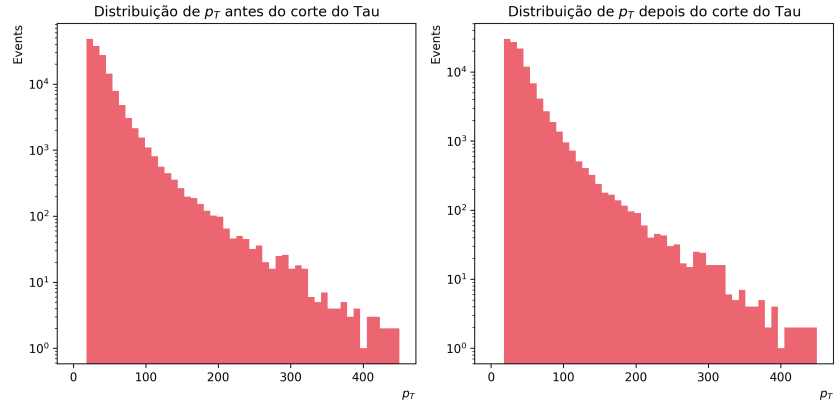
Fonte: O autor.



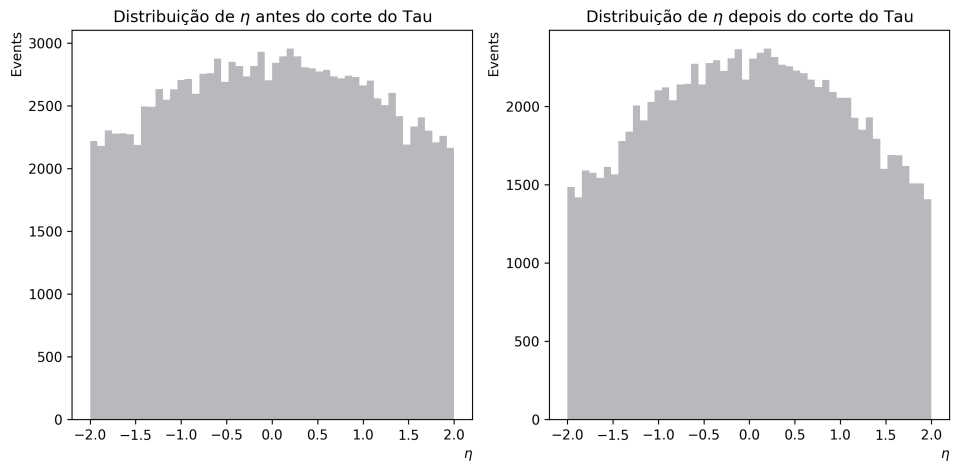
Fonte: O autor.



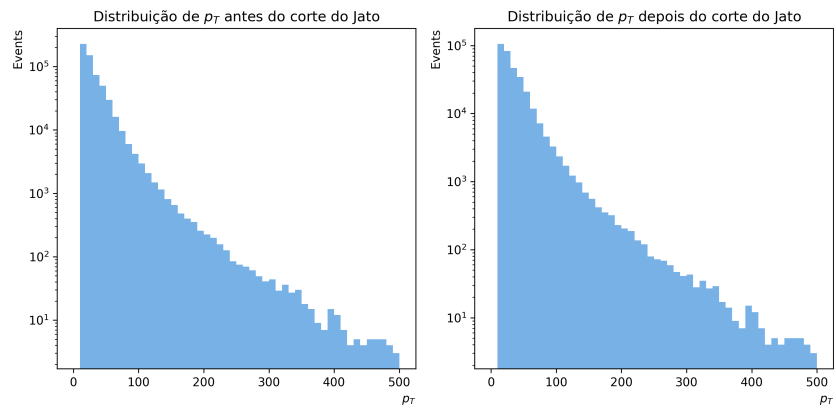
Fonte: O autor.



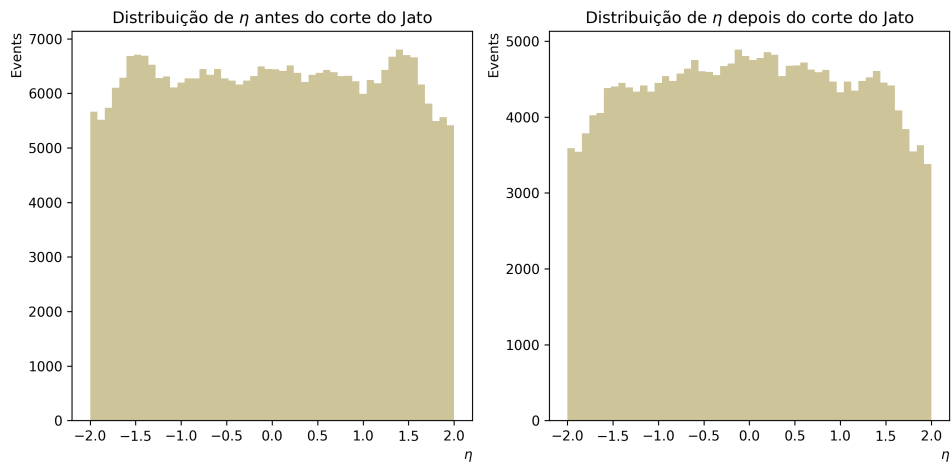
Fonte: O autor.



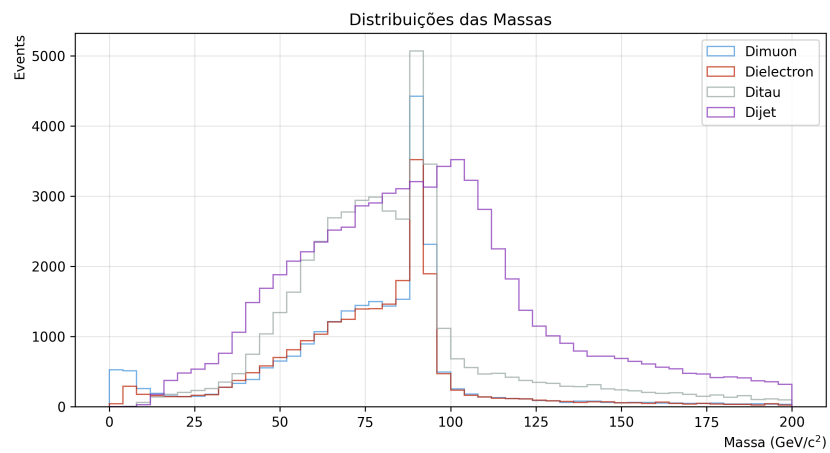
Fonte: O autor.



Fonte: O autor.



Fonte: O autor.



Fonte: O autor.

```

1  import numpy as np
2  import uproot
3  import awkward as ak
4  import matplotlib.pyplot as plt
5  import vector
6  import hist
7  # import mplhep as hep
8  import coffea
9
10
11 path = '/opendata/eos/opendata/cms/mc/RunIISummer20UL16NanoAODv9/ZZTo4L_TuneCP5_13TeV_powheg_pythia8,
12 file = uproot.open(path + "051E9D22-4F30-8E49-8477-644E65768282.root")
13
14
15 path = '/opendata/eos/opendata/cms/mc/RunIISummer20UL16NanoAODv9/ZZTo4L_TuneCP5_13TeV_powheg_pythia8,
16 file = uproot.open(path + "051E9D22-4F30-8E49-8477-644E65768282.root")
17
18 print('Primeiro arquivo:', file1.keys())
19 tree = file["Events;1"]
20 tree.keys()
21
22 # ----- Muons -----
23
24 '''
25 # Selecionar 2 mions
26 valid_muon_pt = ak.num(muon_pt) >= 2
27 '''
28
29 muon_pt = tree['Muon_pt'].array()
30 muon_eta = tree['Muon_eta'].array()
31 muon_phi = tree['Muon_phi'].array()
32 muon_mass = tree['Muon_mass'].array()
33
34
35 valid_muon_pt = ak.num(muon_pt) >= 2
36 mask_muon = (muon_pt[valid_muon_pt][:, 0] > 20) & (np.abs(muon_eta[valid_muon_pt][:, 0] < 2.5))
37 mask_muon2 = (muon_pt[valid_muon_pt][:, 1] > 12) & (np.abs(muon_eta[valid_muon_pt][:, 1] < 2.5))
38 mask_muon_combined = mask_muon & mask_muon2
39
40
41 muon_pt_sel = muon_pt[valid_muon_pt][mask_muon_combined]
42 muon_eta_sel = muon_eta[valid_muon_pt][mask_muon_combined]
43 muon_phi_sel = muon_phi[valid_muon_pt][mask_muon_combined]
44 muon_mass_sel = muon_mass[valid_muon_pt][mask_muon_combined]
45
46
47
48 # ----- Elétrons -----
49
50 electron_pt = tree['Electron_pt'].array()
51 electron_eta = tree['Electron_eta'].array()
52 electron_phi = tree['Electron_phi'].array()
53 electron_mass = tree['Electron_mass'].array()
54
55 # Filtra eventos válidos com pelo menos 2 elétrons
56 valid_electron_pt = ak.num(electron_pt) >= 2
57

```

```

58 # Máscaras para o primeiro elétron
59 mask_electron_pt = electron_pt[valid_electron_pt][:, 0] > 20
60 mask_electron_eta = np.abs(electron_eta[valid_electron_pt][:, 0]) < 2.5
61
62 # Máscaras para o segundo elétron
63 mask_electron_pt2 = electron_pt[valid_electron_pt][:, 1] > 12
64 mask_electron_eta2 = np.abs(electron_eta[valid_electron_pt][:, 1]) < 2.5
65
66 # Máscara combinada
67 mask_electron_combined = mask_electron_pt & mask_electron_eta & mask_electron_pt2 & mask_electron_eta2
68
69
70 # Outra forma de fazer
71 # mask_elec = (electron_pt[valid_events_elec][:, 0] > 12.0) & (np.abs(electron_eta[valid_events_elec][:, 0]) < 2.5)
72 # mask_elec2 = (electron_pt[valid_events_elec][:, 1] > 4.0) & (np.abs(electron_eta[valid_events_elec][:, 1]) < 2.5)
73 # mask_electron_combined = mask_elec & mask_elec2
74
75
76 # Seleção final
77 electron_pt_sel = electron_pt[valid_electron_pt][mask_electron_combined]
78 electron_eta_sel = electron_eta[valid_electron_pt][mask_electron_combined]
79 electron_phi_sel = electron_phi[valid_electron_pt][mask_electron_combined]
80 electron_mass_sel = electron_mass[valid_electron_pt][mask_electron_combined]
81
82
83 # ----- Taus -----
84
85 tau_pt = tree['Tau_pt'].array()
86 tau_eta = tree['Tau_eta'].array()
87 tau_phi = tree['Tau_phi'].array()
88 tau_mass = tree['Tau_mass'].array()
89
90 # Filtra eventos válidos com pelo menos 2 taus
91 valid_tau_pt = ak.num(tau_pt) >= 2
92
93 # Máscaras para o primeiro tau
94 mask_tau_pt = tau_pt[valid_tau_pt][:, 0] > 20
95 mask_tau_eta = np.abs(tau_eta[valid_tau_pt][:, 0]) < 2.5
96
97 # Máscaras para o segundo tau
98 mask_tau_pt2 = tau_pt[valid_tau_pt][:, 1] > 12
99 mask_tau_eta2 = np.abs(tau_eta[valid_tau_pt][:, 1]) < 2.5
100
101 # Máscara combinada
102 mask_tau_combined = mask_tau_pt & mask_tau_eta & mask_tau_pt2 & mask_tau_eta2
103
104 # Seleção final
105 tau_pt_sel = tau_pt[valid_tau_pt][mask_tau_combined]
106 tau_eta_sel = tau_eta[valid_tau_pt][mask_tau_combined]
107 tau_phi_sel = tau_phi[valid_tau_pt][mask_tau_combined]
108 tau_mass_sel = tau_mass[valid_tau_pt][mask_tau_combined]
109
110
111 # ----- Jatos -----
112
113 jet_pt = tree['Jet_pt'].array()
114 jet_eta = tree['Jet_eta'].array()
115 jet_phi = tree['Jet_phi'].array()

```

```

116 jet_mass = tree['Jet_mass'].array()
117
118 # Filtra eventos válidos com pelo menos 2 jatos
119 valid_jet_pt = ak.num(jet_pt) >= 2
120
121 # Máscaras para o primeiro jato
122 mask_jet_pt = jet_pt[valid_jet_pt][:, 0] > 20
123 mask_jet_eta = np.abs(jet_eta[valid_jet_pt][:, 0]) < 2.5
124
125 # Máscaras para o segundo jato
126 mask_jet_pt2 = jet_pt[valid_jet_pt][:, 1] > 12
127 mask_jet_eta2 = np.abs(jet_eta[valid_jet_pt][:, 1]) < 2.5
128
129 # Máscara combinada
130 mask_jet_combined = mask_jet_pt & mask_jet_eta & mask_jet_pt2 & mask_jet_eta2
131
132 # Seleção final
133 jet_pt_sel = jet_pt[valid_jet_pt][mask_jet_combined]
134 jet_eta_sel = jet_eta[valid_jet_pt][mask_jet_combined]
135 jet_phi_sel = jet_phi[valid_jet_pt][mask_jet_combined]
136 jet_mass_sel = jet_mass[valid_jet_pt][mask_jet_combined]
137
138
139 # Remove listas vazias
140
141 # ----- Múons -----
142
143 muon_pt_clean = muon_pt_sel[ak.num(muon_pt_sel) > 0]
144 muon_eta_clean = muon_eta_sel[ak.num(muon_eta_sel) > 0]
145 muon_phi_clean = muon_phi_sel[ak.num(muon_phi_sel) > 0]
146 muon_mass_clean = muon_mass_sel[ak.num(muon_mass_sel) > 0]
147
148
149
150 print('pT:', muon_pt_clean)
151 print('eta:', muon_eta_clean)
152 print('phi:', muon_phi_clean)
153
154
155 # ----- Elétrons -----
156
157 electron_pt_clean = electron_pt_sel[ak.num(electron_pt_sel) > 0]
158 electron_eta_clean = electron_eta_sel[ak.num(electron_eta_sel) > 0]
159 electron_phi_clean = electron_phi_sel[ak.num(electron_phi_sel) > 0]
160 electron_mass_clean = electron_mass_sel[ak.num(electron_mass_sel) > 0]
161
162 # print('Electron pT:', electron_pt_clean)
163 # print('Electron eta:', electron_eta_clean)
164 # print('Electron phi:', electron_phi_clean)
165 # print('Electron mass:', electron_mass_clean)
166
167 # ----- Taus -----
168
169 tau_pt_clean = tau_pt_sel[ak.num(tau_pt_sel) > 0]
170 tau_eta_clean = tau_eta_sel[ak.num(tau_eta_sel) > 0]
171 tau_phi_clean = tau_phi_sel[ak.num(tau_phi_sel) > 0]
172 tau_mass_clean = tau_mass_sel[ak.num(tau_mass_sel) > 0]
173

```

```

174 # print('Tau pT:', tau_pt_clean)
175 # print('Tau eta:', tau_eta_clean)
176 # print('Tau phi:', tau_phi_clean)
177 # print('Tau mass:', tau_mass_clean)
178
179 # ----- Jatos -----
180
181 jet_pt_clean = jet_pt_sel[ak.num(jet_pt_sel) > 0]
182 jet_eta_clean = jet_eta_sel[ak.num(jet_eta_sel) > 0]
183 jet_phi_clean = jet_phi_sel[ak.num(jet_phi_sel) > 0]
184 jet_mass_clean = jet_mass_sel[ak.num(jet_mass_sel) > 0]
185
186 # print('Jet pT:', jet_pt_clean)
187 # print('Jet eta:', jet_eta_clean)
188 # print('Jet phi:', jet_phi_clean)
189 # print('Jet mass:', jet_mass_clean)
190
191
192
193 # Transformando array em 1D, para o plot
194
195 # ----- Múons -----
196
197 muon_pt_flat = ak.flatten(muon_pt_clean)
198 muon_eta_flat = ak.flatten(muon_eta_clean)
199 muon_phi_flat = ak.flatten(muon_phi_clean)
200 muon_mass_flat = ak.flatten(muon_mass_clean)
201
202 # ----- Elétrons -----
203
204 electron_pt_flat = ak.flatten(electron_pt_clean)
205 electron_eta_flat = ak.flatten(electron_eta_clean)
206 electron_phi_flat = ak.flatten(electron_phi_clean)
207 electron_mass_flat = ak.flatten(electron_mass_clean)
208
209 # ----- Taus -----
210
211 tau_pt_flat = ak.flatten(tau_pt_clean)
212 tau_eta_flat = ak.flatten(tau_eta_clean)
213 tau_phi_flat = ak.flatten(tau_phi_clean)
214 tau_mass_flat = ak.flatten(tau_mass_clean)
215
216 # ----- Jatos -----
217
218 jet_pt_flat = ak.flatten(jet_pt_clean)
219 jet_eta_flat = ak.flatten(jet_eta_clean)
220 jet_phi_flat = ak.flatten(jet_phi_clean)
221 jet_mass_flat = ak.flatten(jet_mass_clean)
222
223
224
225 # Quadri-Vetores
226
227 # ----- Múons -----
228
229 muon1_4vecs = vector.zip({
230     "pt": muon_pt_clean[:, 0],
231     "eta": muon_eta_clean[:, 0],

```



```

232     "phi": muon_phi_clean[:, 0],
233     "mass": muon_mass_clean[:, 0]
234 })
235
236 muon2_4vecs = vector.zip({
237     "pt": muon_pt_clean[:, 1],
238     "eta": muon_eta_clean[:, 1],
239     "phi": muon_phi_clean[:, 1],
240     "mass": muon_mass_clean[:, 1]
241 })
242
243 dimuon = muon1_4vecs + muon2_4vecs
244 mass_dimuon = dimuon.mass
245
246 # ----- Elétrons -----
247
248
249 electron1_4vecs = vector.zip({
250     "pt": electron_pt_clean[:, 0],
251     "eta": electron_eta_clean[:, 0],
252     "phi": electron_phi_clean[:, 0],
253     "mass": electron_mass_clean[:, 0]
254 })
255
256 electron2_4vecs = vector.zip({
257     "pt": electron_pt_clean[:, 1],
258     "eta": electron_eta_clean[:, 1],
259     "phi": electron_phi_clean[:, 1],
260     "mass": electron_mass_clean[:, 1]
261 })
262
263 dielectron = electron1_4vecs + electron2_4vecs
264 mass_dielelectron = dielectron.mass
265 # ----- Taus -----
266
267 tau1_4vecs = vector.zip({
268     "pt": tau_pt_clean[:, 0],
269     "eta": tau_eta_clean[:, 0],
270     "phi": tau_phi_clean[:, 0],
271     "mass": tau_mass_clean[:, 0]
272 })
273
274 tau2_4vecs = vector.zip({
275     "pt": tau_pt_clean[:, 1],
276     "eta": tau_eta_clean[:, 1],
277     "phi": tau_phi_clean[:, 1],
278     "mass": tau_mass_clean[:, 1]
279 })
280
281 ditau = tau1_4vecs + tau2_4vecs
282 mass_ditau = ditau.mass
283 # ----- Jatos -----
284
285 jet1_4vecs = vector.zip({
286     "pt": jet_pt_clean[:, 0],
287     "eta": jet_eta_clean[:, 0],
288     "phi": jet_phi_clean[:, 0],
289     "mass": jet_mass_clean[:, 0]

```

```

290 })
291
292 jet2_4vecs = vector.zip({
293     "pt": jet_pt_clean[:, 1],
294     "eta": jet_eta_clean[:, 1],
295     "phi": jet_phi_clean[:, 1],
296     "mass": jet_mass_clean[:, 1]
297 })
298
299
300 dijet = jet1_4vecs + jet2_4vecs
301 mass_dijet = dijet.mass
302
303
304 # ----- Variáveis para os plots antes dos cortes -----
305
306 # ----- Múons -----
307 muon_pt_before_cut = tree['Muon_pt'].array()
308 muon_pt_before_cut_clean = muon_pt_before_cut[ak.num(muon_pt_before_cut) > 0]
309 muon_pt_before = ak.flatten(muon_pt_before_cut_clean)
310
311 muon_eta_before_cut = tree['Muon_eta'].array()
312 muon_eta_before_cut_clean = muon_eta_before_cut[ak.num(muon_eta_before_cut) > 0]
313 muon_eta_before = ak.flatten(muon_eta_before_cut_clean)
314
315 # ----- Electron -----
316 electron_pt_before_cut = tree['Electron_pt'].array()
317 electron_pt_before_cut_clean = electron_pt_before_cut[ak.num(electron_pt_before_cut) > 0]
318 electron_pt_before = ak.flatten(electron_pt_before_cut_clean)
319
320 electron_eta_before_cut = tree['Electron_eta'].array()
321 electron_eta_before_cut_clean = electron_eta_before_cut[ak.num(electron_eta_before_cut) > 0]
322 electron_eta_before = ak.flatten(electron_eta_before_cut_clean)
323
324 # ----- Tau -----
325 tau_pt_before_cut = tree['Tau_pt'].array()
326 tau_pt_before_cut_clean = tau_pt_before_cut[ak.num(tau_pt_before_cut) > 0]
327 tau_pt_before = ak.flatten(tau_pt_before_cut_clean)
328
329 tau_eta_before_cut = tree['Tau_eta'].array()
330 tau_eta_before_cut_clean = tau_eta_before_cut[ak.num(tau_eta_before_cut) > 0]
331 tau_eta_before = ak.flatten(tau_eta_before_cut_clean)
332
333 # ----- Jet -----
334 jet_pt_before_cut = tree['Jet_pt'].array()
335 jet_pt_before_cut_clean = jet_pt_before_cut[ak.num(jet_pt_before_cut) > 0]
336 jet_pt_before = ak.flatten(jet_pt_before_cut_clean)
337
338 jet_eta_before_cut = tree['Jet_eta'].array()
339 jet_eta_before_cut_clean = jet_eta_before_cut[ak.num(jet_eta_before_cut) > 0]
340 jet_eta_before = ak.flatten(jet_eta_before_cut_clean)
341
342
343
344 # //\\//\\//\\//\\ MUON PT //\\//\\//\\//\\
345 plt.figure(figsize=(10, 5))
346
347

```

```

348 plt.subplot(1, 2, 1)
349 plt.subplot(1, 2, 1)
350 plt.hist(muon_pt_before, bins=50, color='#7a21dd', alpha=0.7, range=(0, 1000))
351 plt.yscale('log')
352 plt.title(r'Distribuição de  $p_T$  antes do corte do Múon')
353 plt.xlabel(r" $p_T$ ", loc='right')
354 plt.ylabel("Events", loc='top')
355
356
357 plt.subplot(1, 2, 2)
358 plt.hist(muon_pt_flat, bins=50, color='#7a21dd', alpha=0.7, range=(20, 700))
359 plt.yscale('log')
360 plt.title(r'Distribuição de  $p_T$  depois do corte do Múon')#, loc = 'left')
361 plt.xlabel(r" $p_T$ ", loc='right')
362 plt.ylabel("Events", loc='top')
363
364 plt.tight_layout()
365 plt.show()
366
367
368
369 # /\ /\ /\ /\ /\ /\ /\ MUON ETA /\ /\ /\ /\ /\ /\ /\
370 plt.figure(figsize=(10, 5))
371
372 plt.subplot(1, 2, 1)
373 plt.hist(muon_eta_before, bins=50, color='#5790fc', alpha=0.7, range=(-2, 2))
374 plt.title("Distribuição de  $\eta$  antes do corte do Múon")
375 plt.xlabel(r" $\eta$ ", loc='right')
376 plt.ylabel("Events", loc='top')
377
378 plt.subplot(1, 2, 2)
379 plt.hist(muon_eta_flat, bins=50, color='#5790fc', alpha=0.7, range=(-2, 2))
380 plt.title("Distribuição de  $\eta$  depois do corte do Múon")
381 plt.xlabel(r" $\eta$ ", loc='right')
382 plt.ylabel("Events", loc='top')
383
384 plt.tight_layout()
385 plt.show()
386
387
388 # /\ /\ /\ /\ /\ /\ /\ ELÉTRON PT /\ /\ /\ /\ /\ /\ /\
389 plt.figure(figsize=(10, 5))
390
391 plt.subplot(1, 2, 1)
392 plt.hist(electron_pt_before, bins=50, color='#92dadd', alpha=0.7, range=(0, 600))
393 plt.title(r'Distribuição de  $p_T$  antes do corte do Elétron")
394 plt.xlabel(r" $p_T$ ", loc='right')
395 plt.ylabel("Events", loc='top')
396 plt.yscale('log')
397
398 plt.subplot(1, 2, 2)
399 plt.hist(electron_pt_flat, bins=50, color='#92dadd', alpha=0.7, range=(0, 600))
400 plt.title(r'Distribuição de  $p_T$  depois do corte do Elétron")
401 plt.xlabel(r" $p_T$ ", loc='right')
402 plt.ylabel("Events", loc='top')
403 plt.yscale('log')
404
405 plt.tight_layout()

```

```

406 plt.show()
407
408
409
410 # /\ /\ /\ /\ /\ /\ /\ ELÉTRON ETA /\ /\ /\ /\ /\ /\ /\
411 plt.figure(figsize=(10, 5))
412
413 plt.subplot(1, 2, 1)
414 plt.hist(electron_eta_before, bins=50, color='#f89c20', alpha=0.7, range=(-2, 2))
415 plt.title("Distribuição de  $\eta$  antes do corte do Elétron")
416 plt.xlabel(r" $\eta$ ", loc='right')
417 plt.ylabel("Events", loc='top')
418
419 plt.subplot(1, 2, 2)
420 plt.hist(electron_eta_flat, bins=50, color='#f89c20', alpha=0.7, range=(-2, 2))
421 plt.title("Distribuição de  $\eta$  depois do corte do Elétron")
422 plt.xlabel(r" $\eta$ ", loc='right')
423 plt.ylabel("Events", loc='top')
424
425 plt.tight_layout()
426 plt.show()
427
428
429 # /\ /\ /\ /\ /\ /\ /\ TAU PT /\ /\ /\ /\ /\ /\ /\
430
431 plt.figure(figsize=(10, 5))
432
433 plt.subplot(1, 2, 1)
434 plt.hist(tau_pt_before, bins=50, color='#e42536', alpha=0.7, range=(0, 450))
435 plt.title("Distribuição de  $p_T$  antes do corte do Tau")
436 plt.xlabel(r" $p_T$ ", loc='right')
437 plt.ylabel("Events", loc='top')
438 plt.yscale('log')
439
440 plt.subplot(1, 2, 2)
441 plt.hist(tau_pt_flat, bins=50, color='#e42536', alpha=0.7, range=(0, 450))
442 plt.title("Distribuição de  $p_T$  depois do corte do Tau")
443 plt.xlabel(r" $p_T$ ", loc='right')
444 plt.ylabel("Events", loc='top')
445 plt.yscale('log')
446
447 plt.tight_layout()
448 plt.show()
449
450
451 # /\ /\ /\ /\ /\ /\ /\ TAU ETA /\ /\ /\ /\ /\ /\ /\
452 plt.figure(figsize=(10, 5))
453
454 plt.subplot(1, 2, 1)
455 plt.hist(tau_eta_before, bins=50, color='#9c9ca1', alpha=0.7, range=(-2, 2))
456 plt.title("Distribuição de  $\eta$  antes do corte do Tau")
457 plt.xlabel(r" $\eta$ ", loc='right')
458 plt.ylabel("Events", loc='top')
459
460 plt.subplot(1, 2, 2)
461 plt.hist(tau_eta_flat, bins=50, color='#9c9ca1', alpha=0.7, range=(-2, 2))
462 plt.title("Distribuição de  $\eta$  depois do corte do Tau")
463 plt.xlabel(r" $\eta$ ", loc='right')

```

```

464 plt.ylabel("Events", loc='top')
465
466 plt.tight_layout()
467 plt.show()
468
469
470 # /\ /\ /\ /\ /\ /\ /\ JET PT /\ /\ /\ /\ /\ /\ /\
471 plt.figure(figsize=(10, 5))
472
473 plt.subplot(1, 2, 1)
474 plt.hist(jet_pt_before, bins=50, color='#3f90da', alpha=0.7, range=(0, 500))
475 plt.title("Distribuição de $p_T$ antes do corte do Jato")
476 plt.xlabel(r"$p_T$", loc='right')
477 plt.ylabel("Events", loc='top')
478 plt.yscale('log')
479
480 plt.subplot(1, 2, 2)
481 plt.hist(jet_pt_flat, bins=50, color='#3f90da', alpha=0.7, range=(0, 500))
482 plt.title("Distribuição de $p_T$ depois do corte do Jato")
483 plt.xlabel(r"$p_T$", loc='right')
484 plt.ylabel("Events", loc='top')
485 plt.yscale('log')
486
487 plt.tight_layout()
488 plt.show()
489
490
491 # /\ /\ /\ /\ /\ /\ /\ JET ETA /\ /\ /\ /\ /\ /\ /\
492 plt.figure(figsize=(10, 5))
493
494 plt.subplot(1, 2, 1)
495 plt.hist(jet_eta_before, bins=50, color='#b9ac70', alpha=0.7, range=(-2, 2))
496 plt.title("Distribuição de $\eta$ antes do corte do Jato")
497 plt.xlabel(r"$\eta$", loc='right')
498 plt.ylabel("Events", loc='top')
499
500 plt.subplot(1, 2, 2)
501 plt.hist(jet_eta_flat, bins=50, color='#b9ac70', alpha=0.7, range=(-2, 2))
502 plt.title("Distribuição de $\eta$ depois do corte do Jato")
503 plt.xlabel(r"$\eta$", loc='right')
504 plt.ylabel("Events", loc='top')
505
506 plt.tight_layout()
507 plt.show()
508
509
510 plt.figure(figsize=(10, 5))
511
512 # Plot de cada variável
513 plt.hist(mass_dimuon, bins=50, color='#3f90da', alpha=0.7, label='Dimuon', histtype='step', range =
514 plt.hist(mass_dielectron, bins=50, color='#bd1f01', alpha=0.7, label='Dielectron', histtype='step',
515 plt.hist(mass_ditau, bins=50, color='#94a4a2', alpha=0.7, label='Ditau', histtype='step', range = (0
516 plt.hist(mass_dijet, bins=50, color='#832db6', alpha=0.7, label='Dijet', histtype='step', range = (0
517
518 # Configurações do gráfico
519 plt.title("Distribuições das Massas")
520 plt.xlabel(r"Massa (GeV/c$^2$)", loc='right')
521 plt.ylabel("Events", loc='top')

```

```
522 plt.legend(loc='best')
523 plt.grid(alpha=0.3)
524
525 plt.show()
```
